

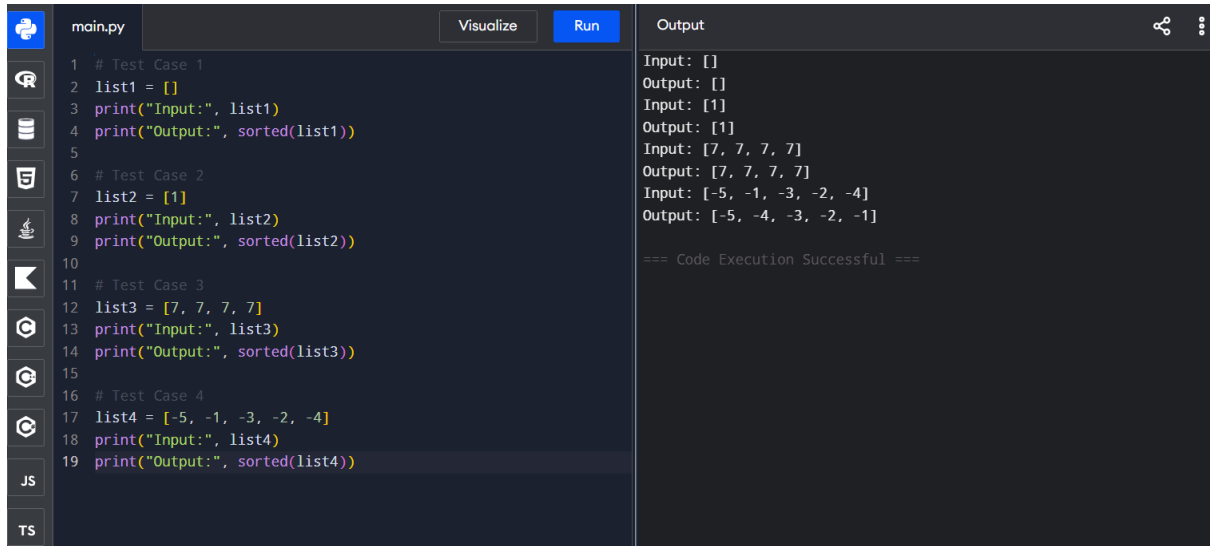
CHAPTER – 2

Name: Jeslet Jessica. T

Regno: 192524135

Course: CSA0603 – DAA

EXPERIMENT – 1

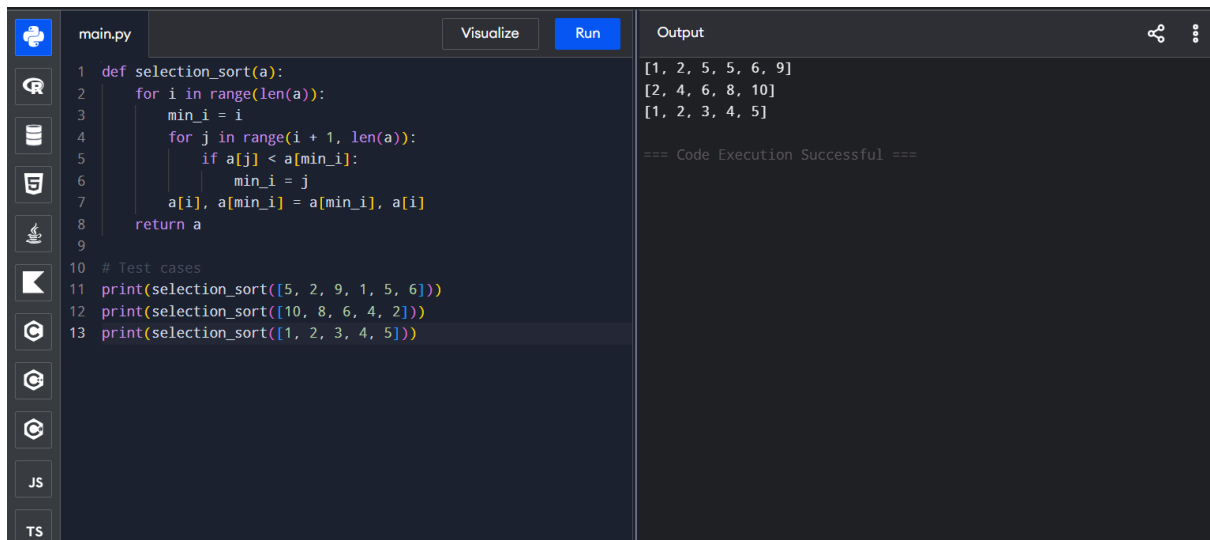


```
main.py Visualize Run Output
1 # Test Case 1
2 list1 = []
3 print("Input:", list1)
4 print("Output:", sorted(list1))
5
6 # Test Case 2
7 list2 = [1]
8 print("Input:", list2)
9 print("Output:", sorted(list2))
10
11 # Test Case 3
12 list3 = [7, 7, 7, 7]
13 print("Input:", list3)
14 print("Output:", sorted(list3))
15
16 # Test Case 4
17 list4 = [-5, -1, -3, -2, -4]
18 print("Input:", list4)
19 print("Output:", sorted(list4))

Input: []
Output: []
Input: [1]
Output: [1]
Input: [7, 7, 7, 7]
Output: [7, 7, 7, 7]
Input: [-5, -1, -3, -2, -4]
Output: [-5, -4, -3, -2, -1]

=== Code Execution Successful ===
```

EXPERIMENT – 2

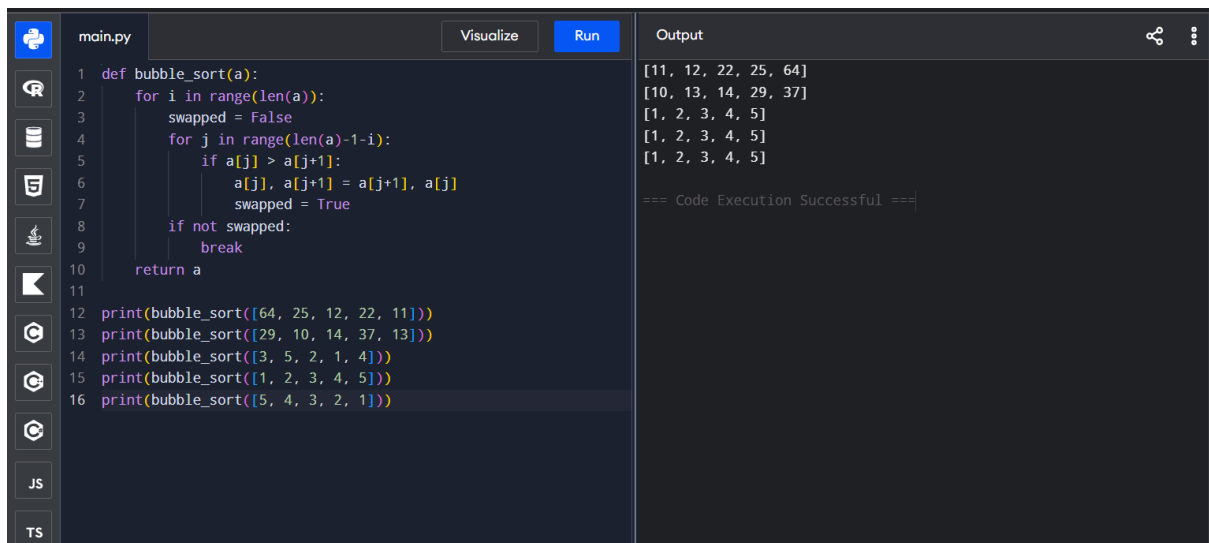


```
main.py Visualize Run Output
1 def selection_sort(a):
2     for i in range(len(a)):
3         min_i = i
4         for j in range(i + 1, len(a)):
5             if a[j] < a[min_i]:
6                 min_i = j
7         a[i], a[min_i] = a[min_i], a[i]
8     return a
9
10 # Test cases
11 print(selection_sort([5, 2, 9, 1, 5, 6]))
12 print(selection_sort([10, 8, 6, 4, 2]))
13 print(selection_sort([1, 2, 3, 4, 5]))

[1, 2, 5, 5, 6, 9]
[2, 4, 6, 8, 10]
[1, 2, 3, 4, 5]

=== Code Execution Successful ===
```

EXPERIMENT – 3



The screenshot shows a Python IDE with a file named `main.py`. The code implements a `bubble_sort` function. The function takes a list `a` and sorts it in ascending order using bubble sort. It uses nested loops: the outer loop iterates from `0` to `len(a)-1`, and the inner loop iterates from `0` to `len(a)-i-1`. If `a[j] > a[j+1]`, the elements are swapped. A `swapped` flag is used to optimize the process, and a `break` statement is used when no swaps occur in a pass. The function returns the sorted list `a`. Below the function, five print statements are used to test the function with different input lists. The output pane on the right shows the results of these tests, which are the sorted versions of the input lists. The output also includes a success message: `=== Code Execution Successful ===`.

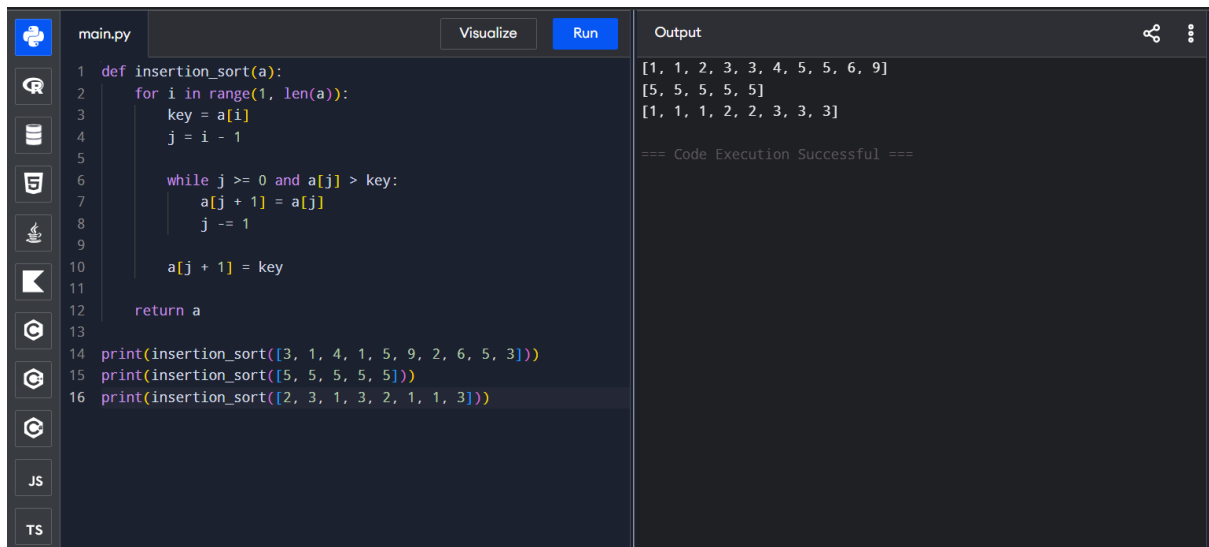
```
1 def bubble_sort(a):
2     for i in range(len(a)):
3         swapped = False
4         for j in range(len(a)-1-i):
5             if a[j] > a[j+1]:
6                 a[j], a[j+1] = a[j+1], a[j]
7                 swapped = True
8         if not swapped:
9             break
10    return a
11
12 print(bubble_sort([64, 25, 12, 22, 11]))
13 print(bubble_sort([29, 10, 14, 37, 13]))
14 print(bubble_sort([3, 5, 2, 1, 4]))
15 print(bubble_sort([1, 2, 3, 4, 5]))
16 print(bubble_sort([5, 4, 3, 2, 1]))
```

Output:

```
[11, 12, 22, 25, 64]
[10, 13, 14, 29, 37]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]

=== Code Execution Successful ===
```

EXPERIMENT – 4



The screenshot shows a Python IDE with a file named `main.py`. The code implements an `insertion_sort` function. The function takes a list `a` and sorts it in ascending order using insertion sort. It iterates from `1` to `len(a)-1`. For each element `a[i]`, it finds its correct position in the sorted subarray `a[0..i-1]` by shifting elements to the right until it finds a smaller element. Then, it inserts `a[i]` at the correct position. The function returns the sorted list `a`. Below the function, three print statements are used to test the function with different input lists. The output pane on the right shows the results of these tests, which are the sorted versions of the input lists. The output also includes a success message: `=== Code Execution Successful ===`.

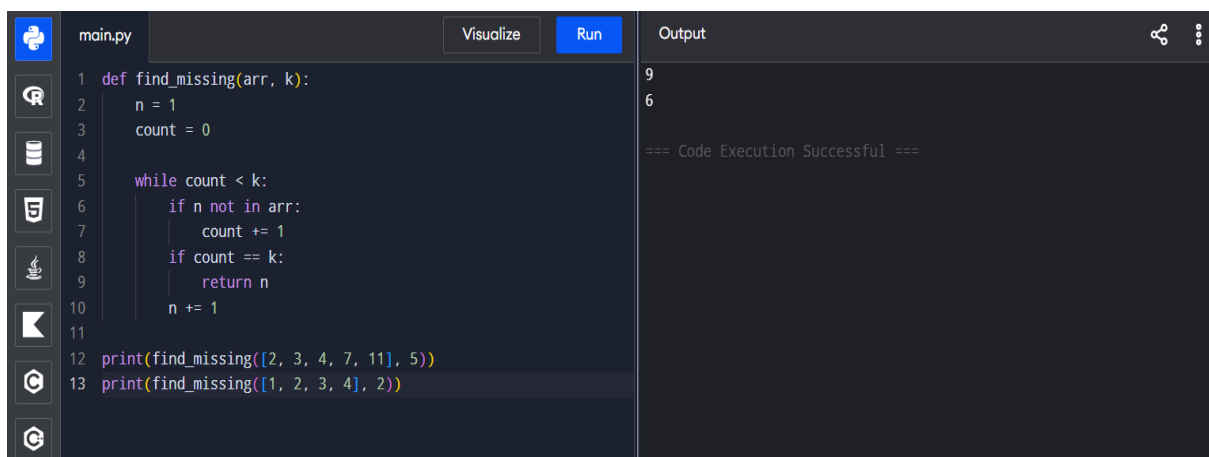
```
1 def insertion_sort(a):
2     for i in range(1, len(a)):
3         key = a[i]
4         j = i - 1
5
6         while j >= 0 and a[j] > key:
7             a[j + 1] = a[j]
8             j -= 1
9
10    a[j + 1] = key
11    return a
12
13
14 print(insertion_sort([3, 1, 4, 1, 5, 9, 2, 6, 5, 3]))
15 print(insertion_sort([5, 5, 5, 5, 5]))
16 print(insertion_sort([2, 3, 1, 3, 2, 1, 1, 3]))
```

Output:

```
[1, 1, 2, 3, 3, 4, 5, 5, 6, 9]
[5, 5, 5, 5, 5]
[1, 1, 1, 2, 2, 3, 3, 3]

=== Code Execution Successful ===
```

EXPERIMENT – 5



The screenshot shows a Python IDE with a file named `main.py`. The code implements a `find_missing` function. The function takes an array `arr` and a number `k` as input. It initializes `n` to `1` and `count` to `0`. It then enters a `while` loop that continues as long as `count < k`. Inside the loop, it checks if `n` is not in `arr`. If it is not, it increments `count` by `1`. If `count` equals `k`, it returns `n`. Otherwise, it increments `n` by `1`. Below the function, two print statements are used to test the function with different input arrays and values of `k`. The output pane on the right shows the results of these tests, which are the missing numbers. The output also includes a success message: `=== Code Execution Successful ===`.

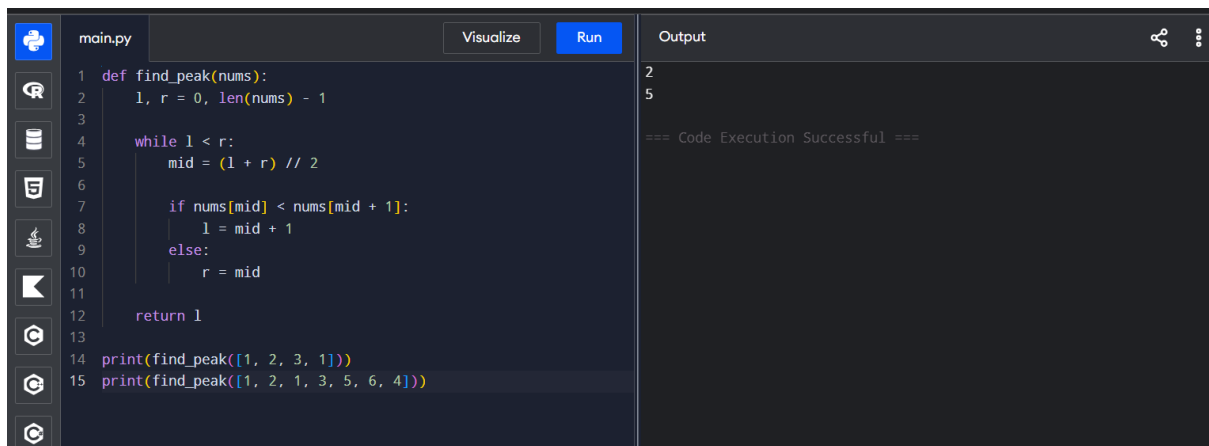
```
1 def find_missing(arr, k):
2     n = 1
3     count = 0
4
5     while count < k:
6         if n not in arr:
7             count += 1
8         if count == k:
9             return n
10    n += 1
11
12 print(find_missing([2, 3, 4, 7, 11], 5))
13 print(find_missing([1, 2, 3, 4], 2))
```

Output:

```
9
6

=== Code Execution Successful ===
```

EXPERIMENT – 6



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'find_peak' that uses a binary search algorithm to find the peak of a list. The function takes a list 'nums' and returns the index of the peak. The code is as follows:

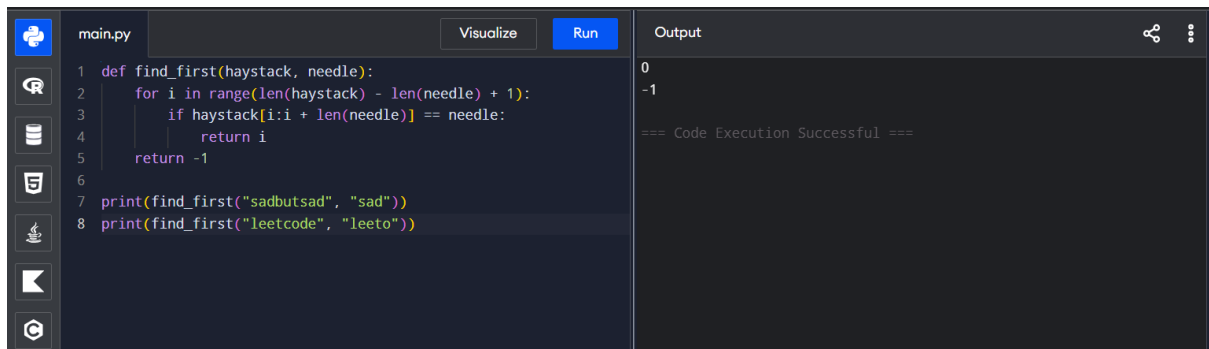
```
1 def find_peak(nums):
2     l, r = 0, len(nums) - 1
3
4     while l < r:
5         mid = (l + r) // 2
6
7         if nums[mid] < nums[mid + 1]:
8             l = mid + 1
9         else:
10            r = mid
11
12    return l
13
14 print(find_peak([1, 2, 3, 1]))
15 print(find_peak([1, 2, 1, 3, 5, 6, 4]))
```

The output of the code is shown on the right side of the editor:

```
2
5

=== Code Execution Successful ===
```

EXPERIMENT – 7



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'find_first' that finds the first occurrence of a needle in a haystack. The function takes a 'haystack' and a 'needle' and returns the index of the first occurrence. The code is as follows:

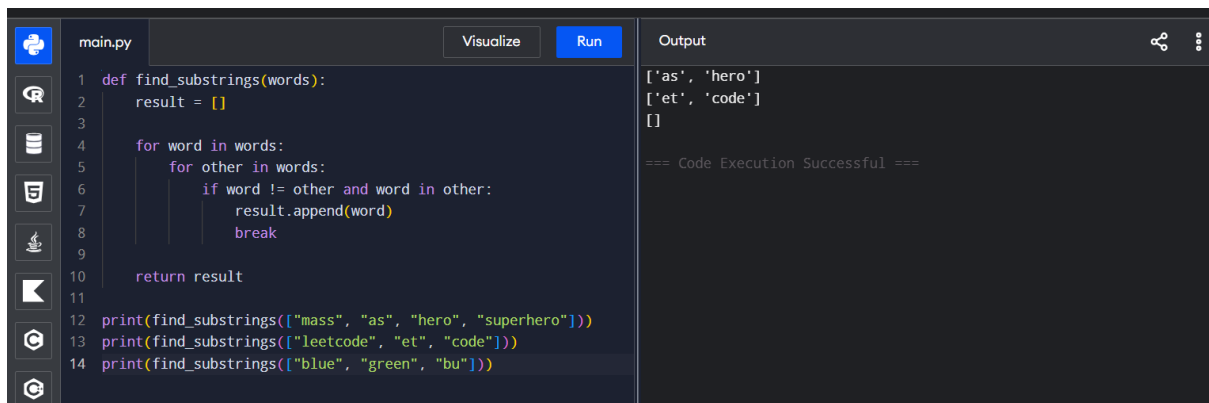
```
1 def find_first(haystack, needle):
2     for i in range(len(haystack) - len(needle) + 1):
3         if haystack[i:i + len(needle)] == needle:
4             return i
5     return -1
6
7 print(find_first("sadbutsad", "sad"))
8 print(find_first("leetcode", "leeto"))
```

The output of the code is shown on the right side of the editor:

```
0
-1

=== Code Execution Successful ===
```

EXPERIMENT – 8



The screenshot shows a code editor with a file named 'main.py'. The code defines a function 'find_substrings' that finds all substrings of a given word. The function takes a list of words and returns a list of all substrings. The code is as follows:

```
1 def find_substrings(words):
2     result = []
3
4     for word in words:
5         for other in words:
6             if word != other and word in other:
7                 result.append(word)
8                 break
9
10    return result
11
12 print(find_substrings(["mass", "as", "hero", "superhero"]))
13 print(find_substrings(["leetcode", "et", "code"]))
14 print(find_substrings(["blue", "green", "bu"]))
```

The output of the code is shown on the right side of the editor:

```
['as', 'hero']
['et', 'code']
[]

=== Code Execution Successful ===
```

EXPERIMENT – 9

main.py	Visualize	Run	Output
<pre>1 import math 2 def closest_pair(points): 3 min_dist = float('inf') 4 pair = () 5 for i in range(len(points)): 6 for j in range(i + 1, len(points)): 7 d = math.sqrt((points[i][0] - points[j][0])**2 + 8 (points[i][1] - points[j][1])**2) 9 if d < min_dist: 10 min_dist = d 11 pair = (points[i], points[j]) 12 return pair, min_dist 13 points = [(1, 2), (4, 5), (7, 8), (3, 1)] 14 pair, distance = closest_pair(points) 15 print("Closest pair:", pair[0], "-", pair[1]) 16 print("Minimum distance:", distance)</pre>			<pre>Closest pair: (1, 2) - (3, 1) Minimum distance: 2.23606797749979 === Code Execution Successful ===</pre>

EXPERIMENT – 10

main.py	Visualize	Run	Output
<pre>1 import math 2 def distance(a, b): 3 return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2) 4 def closest_pair(p): 5 best = float('inf') 6 pair = None 7 for i in range(len(p)): 8 for j in range(i+1, len(p)): 9 d = distance(p[i], p[j]) 10 if d < best: 11 best, pair = d, (p[i], p[j]) 12 return pair, best 13 def convex_hull(p): 14 hull = [] 15 for i in range(len(p)): 16 for j in range(i+1, len(p)): 17 side = [] 18 for k in range(len(p)): 19 if k != i and k != j: 20 x = (p[j][0]-p[i][0])*(p[k][1]-p[i][1]) - 21 \ 22 (p[j][1]-p[i][1])*(p[k][0]-p[i][0])</pre>			<pre>Closest Pair: ((9, 3.5), (7.5, 4.5)) Distance: 1.8027756377319946 Convex Hull: [(10, 0), (5, 3), (15, 3), (6, 6.5), (12.5, 7)] === Code Execution Successful ===</pre>

EXPERIMENT – 11

main.py	Visualize	Run	Output
<pre>1 def convex_hull(points): 2 hull = [] 3 for i in range(len(points)): 4 for j in range(i + 1, len(points)): 5 side = [] 6 for k in range(len(points)): 7 if k != i and k != j: 8 cross = ((points[j][0] - points[i][0]) * 9 (points[k][1] - points[i][1]) - 10 (points[j][1] - points[i][1]) * 11 (points[k][0] - points[i][0])) 12 side.append(cross) 13 if all(x >= 0 for x in side) or all(x <= 0 for x 14 in side): 15 hull += [points[i], points[j]] 16 return list(dict.fromkeys(hull)) 17 points = [(1,1), (4,6), (8,1), (0,0), (3,3)] 18 print("Convex Hull:", convex_hull(points))</pre>			<pre>Convex Hull: [(4, 6), (8, 1), (0, 0)] === Code Execution Successful ===</pre>

EXPERIMENT – 12

main.py	Visualize	Run	Output
<pre>3 def distance(a, b): 4 return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2) 5 def tsp(cities): 6 start = cities[0] 7 best = float('inf') 8 best_path = None 9 for p in permutations(cities[1:]): 10 path = [start] + list(p) + [start] 11 total = 0 12 for i in range(len(path)-1): 13 total += distance(path[i], path[i+1]) 14 if total < best: 15 best = total 16 best_path = path 17 return best, best_path 18 cities1 = [(1,2), (4,5), (7,1), (3,6)] 19 d1, p1 = tsp(cities1) 20 print("Test Case 1:") 21 print("Shortest Distance:", d1) 22 print("Shortest Path:", p1) 23 cities2 = [(2,4), (8,1), (1,7), (6,3), (5,9)] 24 d2, p2 = tsp(cities2) 25 print("\nTest Case 2:")</pre>			Test Case 1: Shortest Distance: 16.969112047670894 Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)] Test Case 2: Shortest Distance: 23.12995011084934 Shortest Path: [(2, 4), (6, 3), (8, 1), (5, 9), (1, 7), (2, 4)] === Code Execution Successful ===

EXPERIMENT – 13

main.py	Visualize	Run	Output
<pre>1 from itertools import permutations 2 def total_cost(a, c): 3 return sum(c[i][a[i]] for i in range(len(a))) 4 def assignment_problem(c): 5 best = float('inf') 6 best_a = None 7 for a in permutations(range(len(c))): 8 x = total_cost(a, c) 9 if x < best: 10 best, best_a = x, a 11 return best_a, best 12 tests = [13 [[3,10,7],[8,5,12],[4,6,9]], 14 [[15,9,4],[8,7,18],[6,12,11]] 15] 16 for i, c in enumerate(tests, 1): 17 a, cost = assignment_problem(c) 18 print("Test Case", i) 19 print("Assignment:", a) 20 print("Total Cost:", cost)</pre>			Test Case 1 Assignment: (2, 1, 0) Total Cost: 16 Test Case 2 Assignment: (2, 1, 0) Total Cost: 17 === Code Execution Successful ===

EXPERIMENT – 14

main.py	Visualize	Run	Output
<pre>1 from itertools import combinations 2 def total_value(items, values): 3 return sum(values[i] for i in items) 4 def is_feasible(items, weights, capacity): 5 return sum(weights[i] for i in items) <= capacity 6 def knapsack(weights, values, capacity): 7 best = [] 8 best_value = 0 9 for r in range(len(weights) + 1): 10 for items in combinations(range(len(weights)), r): 11 if is_feasible(items, weights, capacity): 12 v = total_value(items, values) 13 if v > best_value: 14 best_value = v 15 best = list(items) 16 return best, best_value 17 tests = [18 ([2,3,1], [4,5,3], 4), 19 ([1,2,3,4], [2,4,6,3], 6) 20] 21 for i, (w,v,c) in enumerate(tests, 1): 22 items, value = knapsack(w, v, c)</pre>			Test Case 1 Optimal Selection: [1, 2] Total Value: 8 Test Case 2 Optimal Selection: [0, 1, 2] Total Value: 12 === Code Execution Successful ===